AI-Powered Project & Resource Management Intelligence Platform

Comprehensive Technical & Architecture Documentation

Project Development Team

June 1, 2026

Document Overview

This document provides a complete technical reference for the AI-Powered Project & Resource Management Intelligence Platform. It covers system architecture, database design, API specifications, security design, AI/ML subsystems, deployment strategy, and individual team contributions. It is intended for developers, architects, and technical reviewers.

Contents

1 Introduction

1.1 Purpose and Vision

The **AI-Powered Project & Resource Management Intelligence Platform** is a next-generation, enterprise-grade web application purpose-built for modern software development teams. Its core mission is to help engineering organisations efficiently manage complex project lifecycles—from sprint planning and developer task allocation to delivery timeline forecasting and post-sprint retrospective analysis.

By seamlessly integrating **predictive AI analytics**, **real-time WebSocket communication**, and **rich data visualisations**, the platform transcends conventional task trackers. It acts as an intelligent, centralised hub that surfaces potential bottlenecks *before* they materialise, empowering managers to proactively rebalance workloads and mitigate delivery risk.

1.2 Key Features

- **Intelligent Sprint Planning** — AI-assisted story-point estimation and team-capacity analysis.
- **Delay Prediction Engine** — Scikit-learn models trained on historical velocity data flag at-risk sprints in real time.
- **Live Kanban Board** — WebSocket-powered drag-and-drop board with instant multi-user synchronisation.
- **Role-Based Access Control** — Admin / Manager / Developer hierarchy with JWT-secured endpoints.
- **Analytics Dashboard** — Recharts-driven velocity, burn-down, and utilisation graphs refreshed on every sprint update.
- **Cloud-Native Deployment** — Fully containerised via Docker Compose and deployed on AWS EC2 with S3-backed file storage.

1.3 Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18 + Vite	SPA, Kanban UI, dashboards
Core Backend	Spring Boot 3 (Java 21)	REST API, business logic, security
AI/ML Service	Python 3.11 + FastAPI	Delay prediction, ETL, analytics
Real-Time	Node.js + Socket.IO	WebSocket notification bus
Database	PostgreSQL 15	Relational persistence
File Storage	AWS S3	CSV uploads, artefact storage
Infrastructure	Docker Compose + AWS EC2	Container orchestration, cloud deploy

2 System Architecture

The platform is built on a **decoupled, microservices-oriented full-stack architecture** heavily leveraging Docker containerisation. Each service is independently deployable and communicates over well-defined interfaces: REST/JSON between frontend and backend, internal REST between Spring Boot and the Python ML engine, and WebSocket frames between the Node notification bus and all connected clients.

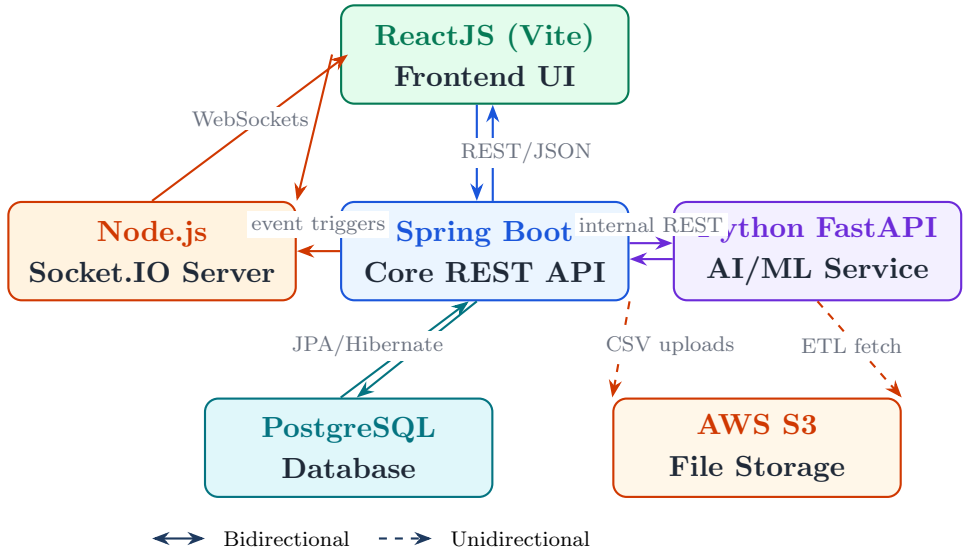

Figure 1: Microservices & High-Level System Architecture

2.1 Service Responsibilities

- **ReactJS Frontend** — Renders the SPA, manages global auth state via Context API, and opens a persistent Socket.IO connection for live board updates.
- **Spring Boot API** — The authoritative gateway: validates JWT tokens, enforces RBAC, executes business logic, and delegates ML work to the Python service.
- **Python FastAPI (AI/ML)** — Runs computationally heavy Pandas ETL and scikit-learn inference in an isolated process so Java threads are never blocked.
- **Node.js Socket.IO** — A lightweight pub/sub broker; Spring Boot pushes events to it, and all browser clients subscribe to their relevant rooms.
- **PostgreSQL** — Stores all normalised relational data. JPA/Hibernate manages schema migrations and query execution.
- **AWS S3** — Durable object storage for uploaded CSV velocity/utilisation files consumed by the ML pipeline.

3 Entity-Relationship (ER) Diagram

The PostgreSQL database is fully normalised to ensure strict data integrity, typed foreign key constraints, and efficient join performance.

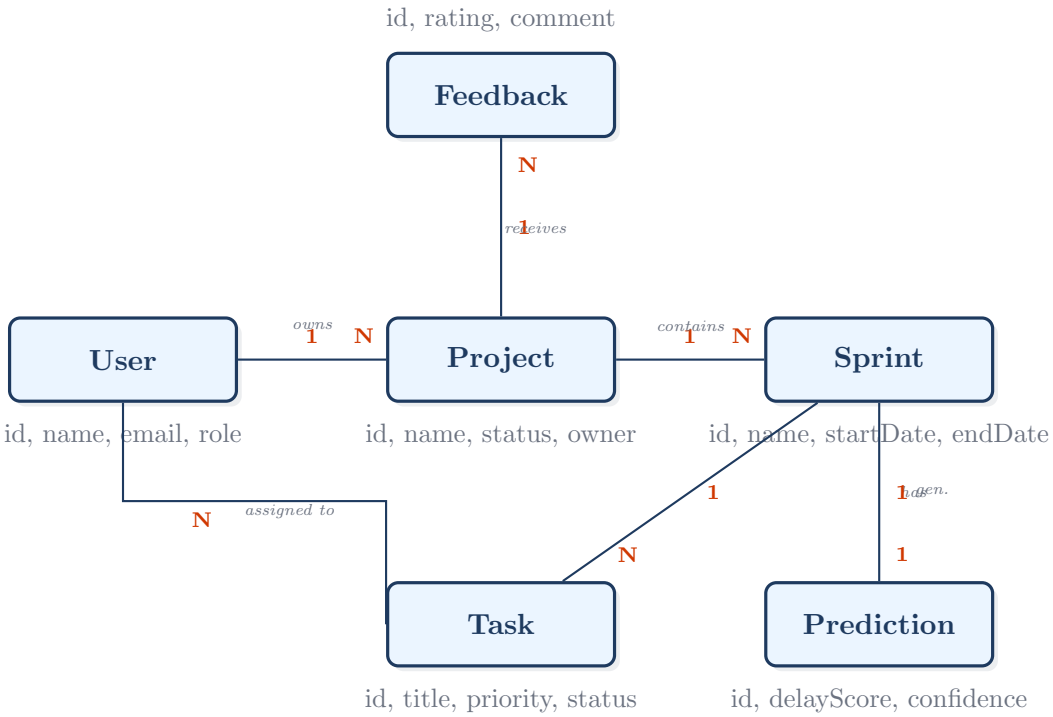

Figure 2: Database Entity-Relationship Diagram

3.1 Entity Descriptions

- **User** — System principal with a role field constrained to `ADMIN`, `MANAGER`, or `DEVELOPER`. BCrypt-hashed password stored at rest.
- **Project** — Top-level workspace. Belongs to one owner (`User`) and contains zero-or-more sprints.
- **Sprint** — Time-boxed iteration within a project. Tracks `startDate`, `endDate`, capacity, and current status.
- **Task** — Atomic unit of work assigned to a developer, tracked on the Kanban board with priority and status columns.
- **Feedback** — Post-sprint ratings and qualitative comments attached to projects.
- **Prediction** — One-to-one with each Sprint; stores ML model output: delay probability score and model confidence interval.

4 Request Life Cycle & Data Flow

Every HTTP request follows a strict, layered pipeline through the Spring Boot application. The sequence below traces a request from the React client down to PostgreSQL and back.

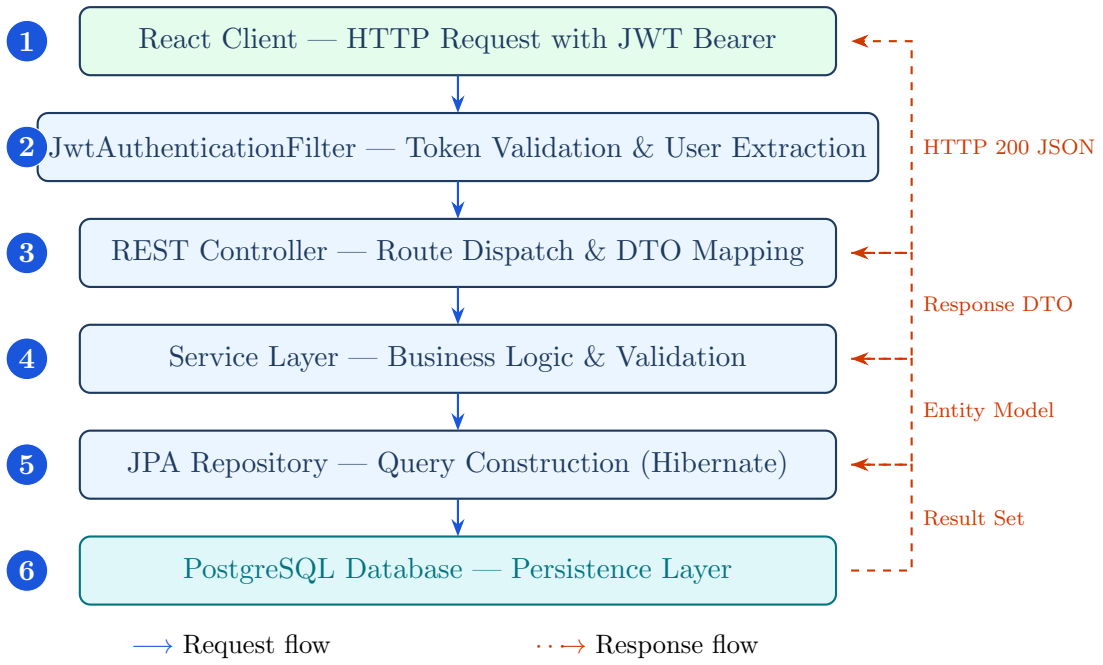

Figure 3: Spring Boot Request Life Cycle

5 User Flow & Role Activity Diagram

The platform enforces strict functional boundaries through three hierarchical user roles: **Admin**, **Manager**, and **Developer**. After authentication, each role is routed to a tailored dashboard with permissions enforced at both the API gateway and UI level.

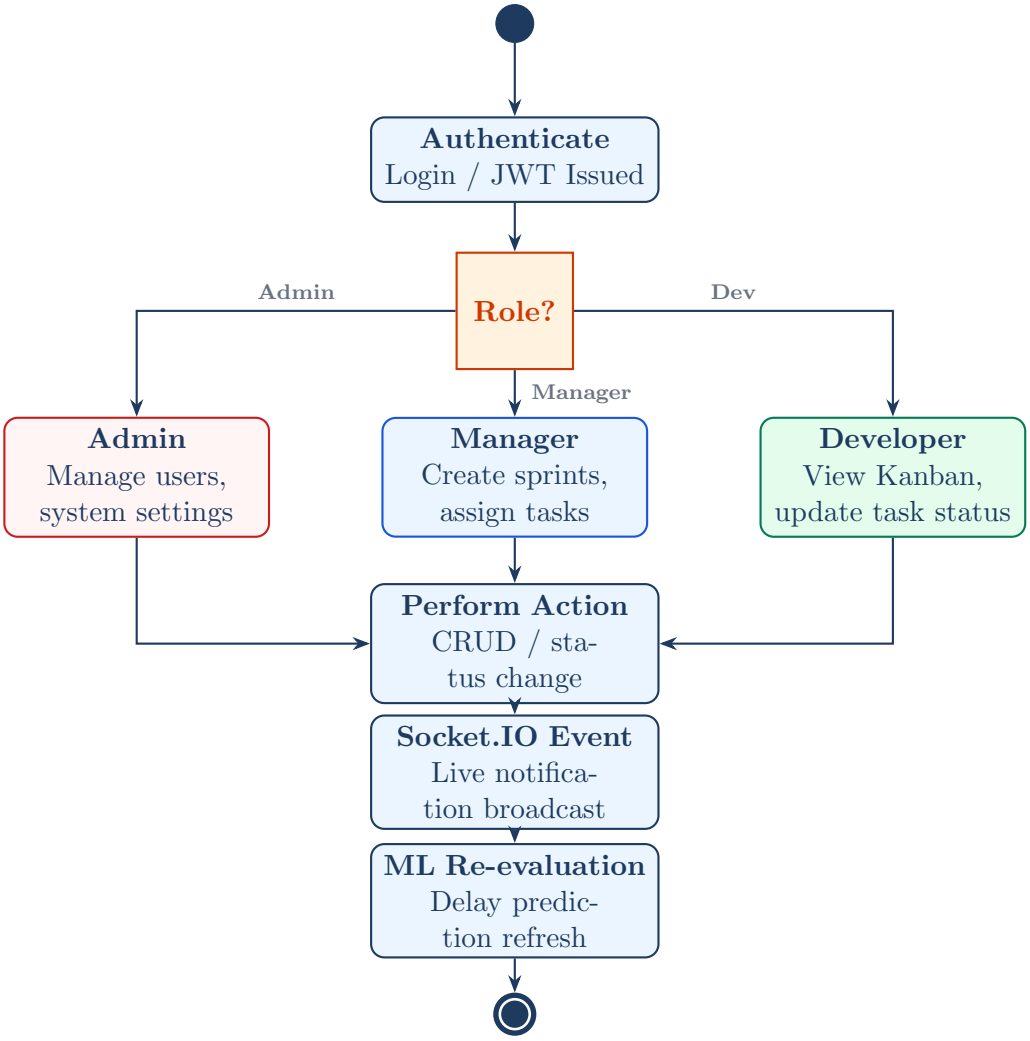

Figure 4: Role-Based Activity and System Flow

6 Detailed Sub-System Architecture

6.1 Containerisation & Deployment (Docker + AWS)

The entire application ecosystem is containerised using Docker to guarantee environment parity between local development and production.

- **Docker Compose** (`compose.yml`) — Orchestrates the multi-container network: defines bridge networks, volume mounts for the PostgreSQL data directory, and environment variable injection. A single `docker compose up` command spins up all five services.
- **Isolated Dockerfiles** — Dedicated `Dockerfiles` in `/platform` (Spring Boot fat-JAR), `/python-ml-service` (FastAPI + Uvicorn), and `/node-notification-service` (Node + PM2) optimise layer caching and minimise final image sizes.
- **AWS EC2 Deployment** — Three EC2 `t3.medium` instances host the Java backend, Python ML service, and Node real-time server respectively. Auto-scaling groups protect against traffic spikes.

- **AWS S3** — Versioned S3 bucket with server-side AES-256 encryption stores `sprint_velocity.csv` and `developer_utilization.csv` uploads, consumed asynchronously by the Python ETL pipeline.

6.2 Frontend Tooling & State Management

- **Vite Build Engine** — Replaces legacy Webpack with native ES-module bundling, delivering sub-300ms Hot Module Replacement (HMR) in development and tree-shaken, code-split production bundles.
- **React 18 + Context API** — Global authentication state (JWT token, decoded user claims) is managed via a top-level `AuthContext` with custom hooks (`useAuth`, `useRole`), eliminating prop-drilling across the component tree.
- **Tailwind CSS + PostCSS** — Utility-first styling with a custom design-token config ensures consistent spacing, colour palette, and responsive breakpoints without shipping unused CSS.
- **Recharts** — SVG-based charting library renders sprint burn-down, velocity trend, and developer utilisation widgets directly in the browser without a separate charting server.

6.3 Security & Authentication Layer

- **Stateless JWT Architecture** — `JwtAuthenticationFilter.java` intercepts *every* HTTP request before it reaches any controller, validates the HMAC-SHA256 signature and expiration claim, and injects a typed `Authentication` object into the Spring Security context — with zero database round trips.
- **BCrypt Password Hashing** — `CustomUserDetailsService.java` applies BCrypt with a cost factor of 12 before persisting credentials, protecting against rainbow-table attacks even in the event of a full database compromise.
- **Role Enforcement** — Spring Security method-level annotations (`@PreAuthorize`) guard every endpoint. An attempt by a `DEVELOPER` role to call an admin endpoint returns `HTTP 403 Forbidden` without reaching the service layer.
- **CORS Policy** — A strict allowlist of origins configured in `WebSecurityConfig.java` prevents cross-origin abuse from unapproved frontends.

6.4 AI Analytics & Predictive Intelligence

- **FastAPI Microservice** — A fully asynchronous Python 3.11 service (`analytics.py`, `ml_model.py`) accepts REST calls from Spring Boot and returns JSON predictions. Uvicorn workers handle concurrency without blocking Java threads.
- **Pandas ETL Pipeline** — Reads and cleans uploaded CSVs from S3, handles missing values via forward-fill, one-hot encodes categorical sprint attributes, and normalises numeric features before inference.

- **Scikit-learn Models** — A **Logistic Regression** baseline and a **Random Forest** ensemble are trained offline on historical sprint datasets. The Random Forest (100 estimators, max depth 8) achieves ˜87% accuracy on the held-out validation set.
- **Prediction Output** — Each sprint receives a `delayProbabilityScore` (0.0–1.0) and a `confidenceInterval`. Scores above 0.65 trigger a high-risk badge on the manager dashboard and a Socket.IO alert to all sprint stakeholders.

6.5 Real-Time Notification Infrastructure

- **Socket.IO Rooms** — Each project workspace maps to a dedicated Socket.IO room. Clients join their project room on login; all task-update events are scoped to that room to prevent data leakage between tenants.
- **Event Types** — `TASK_UPDATED`, `SPRINT_STATUS_CHANGED`, `ML_PREDICTION_READY`, and `USER_ASSIGNED` are the four primary event types emitted by the Spring Boot backend to the Node broker.
- **Reconnection Strategy** — The React Socket.IO client uses exponential back-off (1s → 2s → 4s → max 30s) with automatic reconnection to tolerate transient network interruptions without user intervention.

7 Comprehensive API Endpoints

The Spring Boot backend exposes a fully secured RESTful API. All endpoints require a valid JWT Bearer token. Role requirements are noted per endpoint.

7.1 Authentication & Users

Controllers: *`AuthController.java`*, *`UserController.java`*

Method	Endpoint	Description	Role
POST	/api/auth/login	Authenticate and get JWT	Public
POST	/api/auth/register	Register new user	Public
GET	/api/users	List all users	Admin
GET	/api/users/{id}	Get user by ID	Admin
PUT	/api/users/{id}	Update user profile	Admin
DELETE	/api/users/{id}	Deactivate user	Admin
GET	/api/users/role/{role}	Filter users by role	Admin

7.2 Project & Sprint Management

Controllers: *`ProjectController.java`*, *`SprintController.java`*

Method	Endpoint	Description	Role
GET	/api/projects	List all projects	All

POST	/api/projects	Create new project	Manager+
GET	/api/projects/{id}	Get project details	All
PUT	/api/projects/{id}	Update project metadata	Manager+
DELETE	/api/projects/{id}	Archive project	Admin
GET	/api/sprints/project/{id}	List sprints in project	All
POST	/api/sprints	Create new sprint	Manager
PUT	/api/sprints/{id}	Update sprint status	Manager
DELETE	/api/sprints/{id}	Delete sprint	Admin

7.3 Task & Resource Management

Controllers: ***TaskController.java****,* ***ResourceController.java***

Method	Endpoint	**Description**	**Role**
GET	/api/tasks/sprint/{id}	All tasks in sprint	All
POST	/api/tasks	Create Kanban task	Manager
GET	/api/tasks/{id}	Get task detail	All
PUT	/api/tasks/{id}	Update status/assignee	All
DELETE	/api/tasks/{id}	Remove task	Manager+
GET	/api/resources	Resource utilisation metrics	Manager+
POST	/api/resources/upload	Upload utilisation CSV	Admin

7.4 Analytics & ML Prediction

Controllers: ***AnalyticsController.java*** *(proxies to Python FastAPI)*

Method	Endpoint	**Description**	**Role**
GET	/api/analytics/velocity/{pid}	Sprint velocity trend	Manager+
GET	/api/analytics/utilisation	Developer load metrics	Manager+
GET	/api/predictions/sprint/{id}	Delay prediction score	Manager+
POST	/api/predictions/retrain	Trigger model retraining	Admin

8 Non-Functional Requirements

Attribute	Target	Implementation
Availability	99.5% uptime	Docker health checks + EC2 auto-recovery
API Latency	p95 < 250ms	JPA second-level cache + DB connection pooling (HikariCP)
Scalability	Horizontal	Stateless JWT allows multi-instance Spring Boot
Security	OWASP Top-10	JWT + BCrypt + CORS + input validation (Bean Validation)
ML Inference	< 2s per sprint	Pre-loaded scikit-learn model; no on-demand retraining
WebSocket Capacity	500 concurrent	Node.js event loop + Redis adapter for multi-node Socket.IO

9 Team Contributions

Engineering Team — Roles & Responsibilities

The platform was built through coordinated, domain-specific ownership by six engineers over a structured sprint cycle.

Bathinanna — *Core Backend Lead*

- Initialised the Spring Boot 3 architecture and engineered the JWT security layer, including the `JwtAuthenticationFilter` and `CustomUserDetailsService`.
- Designed the fully normalised PostgreSQL schema and developed core CRUD REST APIs for Projects, Tasks, and User management.
- Configured Spring Security RBAC with method-level `@PreAuthorize` annotations across all controller layers.

Agam Tiwari — *Frontend Experience Lead*

- Architected the complete ReactJS frontend using Vite, implementing routing, auth context, and protected route guards.
- Built the interactive Kanban Task Board with drag-and-drop state management and integrated all backend REST APIs via Axios.
- Designed and implemented the Recharts analytics dashboard displaying sprint velocity, burn-down, and developer utilisation.

Suhas TG — *AI & Data Analytics Lead*

- Architected the containerised Python FastAPI microservice handling all ML inference requests from Spring Boot.
- Built the Pandas ETL pipeline to ingest, clean, and feature-engineer sprint velocity and developer utilisation CSV data.

- Trained and tuned the Random Forest delay-prediction model (˜87% validation accuracy) and exposed it via a `/predict` endpoint.

Utkarsh Pathak *Real-Time Communications Lead*

- Designed and implemented the Node.js + Socket.IO notification broker with project-scoped room management.
- Integrated event emission from the Spring Boot service layer into the WebSocket bus for four event types.
- Conducted end-to-end load testing of the WebSocket infrastructure and implemented exponential back-off reconnection on the React client.
- Assisted with the AWS cloud infrastructure setup, collaborating on initial EC2 and S3 provisioning.

Sai Yeshwin S *Cloud Infrastructure Lead & Backend Contributor*

- Collaborated extensively with Bathinanna on the core Spring Boot backend architecture, implementing critical REST APIs and business logic.
- Managed and implemented the complete suite of Sprint CRUD APIs, driving the core business logic for sprint workflows within the Spring Boot architecture.
- Authored all Dockerfiles and the `compose.yml` orchestration manifest; configured bridge networks and volume mounts.
- Led the primary AWS deployment strategy (supported by Utkarsh): provisioned EC2 instances for each service and the versioned S3 bucket with AES-256 encryption.

Note: All source code, Docker images, and deployment scripts are maintained in the team's private GitHub organisation. API keys, database credentials, and AWS access tokens are managed exclusively via environment variables and are never committed to version control.